Dev2 MVP 실행 학습 노트

날짜: 2026-05-29 복습 예정: D+1, D+3, D+7

한 줄 요약

Docker postgres 켜고 → FastAPI 서버 띄우고 → curl로 엔드포인트 호출하면 MVP 동작.

전체 실행 흐름

```
docker compose up -d postgres
  → 컨테이너 시작 + migrations/ 폴더 SQL 자동 실행 → 테이블 생성

uvicorn dev2.main:app --port 8005
  → FastAPI 서버 기동 → /scan, /insights, /issues, /health 엔드포인트 활성화

POST /scan/mock-bandit
  → mock_vulnerable/ 폴더 Bandit 스캔
  → bandit_result.json 파싱
  → db.py 함수로 PostgreSQL INSERT
  → 결과 JSON 반환

GET /insights/developer/{id}
  → DB에서 scan_issues 집계 → 통계 반환
```

실제 명령어 (복붙용)

1. postgres 시작

```
cd sub2_new/dev2

# Windows에서 필수 (Docker Desktop이 아닌 default context 사용)
docker context use default

docker compose up -d postgres

# 테이블 확인
docker exec dev2-postgres-1 psql -U dev2 -d dev2_db -c "\dt"
```

2. FastAPI 서버 시작

```
cd sub2_new

DATABASE_URL="postgresql+asyncpg://dev2:dev2@localhost:5432/dev2_db" \
USE_MOCK_LLM=true \
ANTHROPIC_API_KEY=sk-ant-dummy \
LANGCHAIN_TRACING_V2=false \
python -m uvicorn dev2.main:app --port 8005
```

포트 8002/8003은 다른 팀원 서비스가 점유 중 → 8005 사용

3. 헬스 체크

```
curl http://localhost:8005/health
# {"status":"ok","service":"dev2"}
```

4. Demo 시나리오 실행 **(Prototype 2)**

```
curl -X POST http://localhost:8005/scan/mock-bandit
# {"status":"success","scan_id":4,"issue_count":10,"summary":
{"critical":2,"major":3,"minor":5}}
```

5. 결과 조회

```
# API로 조회
curl http://localhost:8005/insights/developer/dev2
curl http://localhost:8005/insights/team

# DB 직접 조회
docker exec dev2-postgres-1 psql -U dev2 -d dev2_db \
  -c "SELECT severity, rule_id, message FROM scan_issues ORDER BY severity,
line;"
```

6. 정식 **/scan** 엔드포인트 **(Prototype 1)**

```
curl -X POST http://localhost:8005/scan \
  -H "Content-Type: application/json" \
  -d '{
    "task_id": "test-001",
    "pr_id": "PR-42",
    "pr_diff": "diff --git a/auth.py b/auth.py\n...",
    "developer": {
      "id": "dev1",
      "skill_level": "junior",
      "tech_stack": ["python"]
    },
    "context": {
      "architecture_docs": [],
      "coding_conventions": "",
      "previous_pr_issues": []
    }
  }'
# → 202 Accepted (LangGraph 5노드 파이프라인 백그라운드 실행)
```

주요 트러블슈팅

증상	원인	해결
`dockerDesktopLinuxEngine` 에러	Docker context가 desktop-linux	`docker context use default`
`password authentication failed`	.env와 실제 볼륨 비밀번호 불일치	`POSTGRES_PASSWORD=dev2` (볼륨 초기화 시점 값)
서버 올렸는데 `/feeds/search` 라우트	다른 팀원 서비스가 포트 점유	다른 포트(8005) 사용
`PYTHONPATH` 없으면 `dev2` 모듈 못 찾음	Python이 `sub2_new/dev2`를 패키지로 인식 못함	`PYTHONPATH=/c/.../sub2_new` 설정

Swagger UI

`http://localhost:8005/docs`

브라우저에서 모든 엔드포인트 GUI 테스트 가능.

핵심 개념 3가지

1. 마이그레이션 자동 실행: `docker-compose.yml`의 `volumes: ./migrations:/docker-entrypoint-initdb.d` → postgres 최초 시작 시 SQL 파일 자동 실행
2. **202 Accepted** 패턴: POST /scan은 즉시 응답 반환 후 BackgroundTasks로 무거운 LLM 분석 실행 (클라이언트 타임아웃 방지)
3. **USE_MOCK_LLM=true**: LLM API 없이도 전체 파이프라인 테스트 가능 (개발 단계)

복습 퀴즈

1. Q: postgres 컨테이너가 테이블을 자동 생성하는 이유는? A: (docker-compose의 migrations 볼륨 마운트 → initdb.d 자동 실행)
2. Q: POST /scan이 202를 반환하는 이유는? A: (LLM 분석이 수십 초 걸려 클라이언트 타임아웃 방지 위해 BackgroundTasks 패턴 사용)
3. Q: Windows에서 Docker 사용 시 먼저 해야 할 것은? A: (`docker context use default` — desktop-linux context는 Docker Desktop 전용)